

SIMATS ENGINEERING
ASSESSMENT TOOL 3
Annexure A – Reverse Engineering

Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO5
Description	Design AI-based systems using PySpark, RDD operations, and intelligent agent
Assessment Weightage	50%
Duration	1 Hour
Topic Chosen	Reverse Engineering of APIs Using AI

Submitted by

Name: S. KARTHIKEYAN

Register No: 192419048

Code of Conduct

I, S. Karthikeyan (Reg. No: 192419048), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Reverse Engineering of APIs Using AI

Activity Brief: APIs may need to be understood when documentation is incomplete or unavailable. This report analyses request-response structure and observed behaviour of a test API, uses AI to infer endpoint functionality, generates documentation from that inference, and compares the AI-generated documentation with the actual API specification.

1. Problem Identification

Many real-world integrations rely on third-party or legacy APIs whose formal documentation is missing, outdated, or incomplete – a common situation when working with internal microservices, undocumented mobile-app backends, or deprecated public APIs. In such cases, developers must reconstruct an understanding of what an API does purely from its observable behaviour: the requests it accepts and the responses it returns. This is a genuine reverse-engineering problem, since the underlying implementation is a black box and only the external interface (inputs/outputs) can be observed. The specific engineering question this report investigates is: **can an AI system, given only a sample of real request-response traffic, infer accurate endpoint documentation (purpose, parameters, data types, and relationships) without access to the API's source code or official specification?**

2. Technical Content: How AI Assists API Reverse Engineering

AI-assisted API reverse engineering typically combines three techniques:

- **Traffic capture and structuring:** Tools such as Postman, Insomnia, or a man-in-the-middle proxy (e.g., mitmproxy, Charles Proxy) record the raw HTTP requests and responses exchanged with the API, capturing method (GET/POST/etc.), URL path, headers, query/body parameters, and JSON/XML response bodies.
- **Schema and pattern inference:** The captured request-response pairs are parsed to infer a schema – field names, data types (string/int/boolean/array/object), which fields are optional versus always-present, and value ranges/enums observed across multiple samples of the same endpoint.
- **LLM-based natural-language documentation generation:** A large language model is prompted with a small set of representative request/response examples for an endpoint and asked to (a) describe the endpoint's likely purpose, (b) list and describe each parameter/field with its inferred type, and (c) identify relationships between endpoints (e.g., an `id` returned by one endpoint being used as a path parameter in another) – effectively converting structured examples into human-readable API documentation such as an OpenAPI/Swagger description.

This combination is powerful because pattern inference alone produces a correct but purely structural schema (field names and types), while the LLM step adds semantic interpretation (what a field or endpoint actually *means*), which is the harder and more valuable part of documentation recovery.

3. Use of Tools – Methodology

The following tools and steps were used for this exercise:

- **Test/reference API:** a documented, freely accessible test REST API is used so that AI-generated documentation can be checked against a known-correct specification.
- **Traffic capture tool:** Postman (or curl) to send sample requests to each endpoint and record raw JSON responses.

- **Data structuring:** Python with the `requests` and `pandas` libraries to organise captured request/response pairs into a small structured dataset (one row per sample call).
- **AI documentation generation:** an LLM (e.g., via chat interface or API) is given 2–3 sample request/response pairs per endpoint and prompted to produce an endpoint description, a parameter table, and an inferred data model.
- **Comparison:** the AI-generated documentation is checked line-by-line against the API's real published reference documentation to identify matches, omissions, and hallucinated (incorrect) details.

4. Worked Illustrative Example (Reference Walkthrough)

To illustrate the exact format expected, this section walks through the method using **JSONPlaceholder** (<https://jsonplaceholder.typicode.com/>), a free, publicly documented fake REST API commonly used for prototyping and testing, which exposes resources such as /posts, /comments, /users, and /todos. Because its specification is public, it is ideal for demonstrating the comparison step.

Step 1 – Capture a sample request/response

```
GET https://jsonplaceholder.typicode.com/posts/1
```

Response 200:

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio",
  "body": "quia et suscipit ... reprehenderit"
}
```

Step 2 – Prompt an AI model with the sample

A prompt such as the following is given to an LLM: *“Here is a sample request and JSON response from an unknown API endpoint. Infer the endpoint's purpose, list each field with its data type, and state which fields look like identifiers or foreign keys.”* followed by the captured JSON above (and, ideally, 2–3 more samples from different post IDs so the model can confirm which fields vary and which stay structurally consistent).

Step 3 – Example AI-generated documentation

Endpoint: GET /posts/{id}

Purpose : Retrieve a single blog-post-like resource by its numeric ID.

Parameters:

id (path, integer, required) - unique identifier of the post

Response fields:

userId (integer) - identifier of the post's owning user (likely FK -> /users/{id})

id (integer) - unique identifier of this post

title (string) - short post title

body (string) - main post text content

Step 4 – Compare against the actual specification

Aspect	AI-Generated Doc	Actual JSONPlaceholder Spec	Match?
Endpoint purpose	Retrieve a single post resource	Returns one post resource by id	Yes
userId field	Identified as likely FK to /users	Confirmed relationship to /users resource	Yes
Data types	All four fields typed correctly	Matches actual JSON schema	Yes
Write operations (POST, PUT, DELETE)	Not included	Supported but only simulated (no real persistence)	No

This worked example shows the expected format only. **JSONPlaceholder is used purely as a reference walkthrough** because its specification is public and well known, which makes the comparison step easy to verify; the actual assignment requires you to repeat this process on your own selected API.

5. Your Own API – Results (To Be Completed)

[Fill in with your own work] State which API you selected, why (documented/open/controlled test API), and its base URL.

API selected: _____

Endpoint(s) analysed: _____

5.1 Captured Request/Response Samples

[Fill in with your own work] Paste 2–3 real request/response pairs you captured (via Postman/curl) for each endpoint you analysed, in the same format shown in Section 4, Step 1.

5.2 AI-Generated Documentation

[Fill in with your own work] Paste the actual documentation your AI tool generated from your captured samples, in the same format as Section 4, Step 3.

5.3 Comparison Table

[Fill in with your own work] Fill in this table the same way as the JSONPlaceholder example, using your own API's real published/observed specification as the ground truth.

Aspect	AI-Generated Doc	Actual API Spec	Match?

6. Critical Evaluation – Effectiveness and Limitations of AI Here

Based on this workflow (and to be confirmed against your own results in Section 5), AI-assisted API reverse engineering is generally **effective for structural inference** – correctly identifying field names, data types, and simple foreign-key-like relationships from a handful of samples – because this pattern is directly visible in the JSON structure itself. It is **less reliable for behavioural or write-side inference**: an AI model shown only GET responses cannot know how a POST/PUT/DELETE endpoint will validate input, what error codes it returns for invalid data, or undocumented side effects (e.g., rate limits, authentication scopes), since none of that is observable from read-only samples. It can also **'hallucinate' plausible-sounding but incorrect field meanings** when a field name is ambiguous (e.g., guessing a generic status field represents order status when it actually represents publication status), which is why the comparison-against-ground-truth step in Section 5.3 is essential rather than optional.

7. Expected Outcome

This exercise demonstrates that AI can substantially accelerate the first-pass documentation of an undocumented or partially documented API by converting observed request/response traffic into

structured, readable documentation, but that its output should be treated as a **draft requiring human verification** rather than an authoritative specification, particularly for anything not directly observable in the captured samples (write-behaviour, error handling, authentication, and rate-limiting rules).

8. Notes for Presentation & Delivery

- Lead with the problem (undocumented APIs are common and costly to integrate with) before showing the method.
- Live-demo the capture → AI prompt → generated-doc → comparison flow on your own chosen API if time permits – this maps directly to the 'Use of Tools' rubric criterion.
- Be ready to explain in your own words why the AI got any field wrong in your comparison table — this demonstrates critical evaluation, not just tool use.
- Keep slides to one idea per slide (problem, method, worked example, your results, limitations, conclusion) and rehearse timing to stay within the session's duration.